Design Studio — Feature Requirements

Document Information

- **Version:** 1.0
- **Date:** April 2026
- **Owner:** Nex-Gen LED LLC — Product Team
- **Product:** Nex-Gen Lumina v1.6
- **Feature:** Design Studio — custom LED configuration and design system

---

1. Executive Summary

Design Studio is Lumina's premium customization surface — the feature that lets a customer craft a lighting design fitted to the unique shape of their home. It bridges the gap between generic lighting patterns and genuinely personalized illumination by giving the app a map of the customer's roofline: where each LED sits, how segments connect, where the peaks and corners are. Nex-Gen LED LLC ships permanent residential and commercial lighting that works as hard as the people inside, and Design Studio is what lets the lighting tell their story specifically.

---

2. Core Requirements

2.1 LED Channel Mapping & Identification

REQ-2.1.1: Channel Start/End Definition

- **Priority:** P0 (Critical)
- **Description:** The system SHALL allow the customer to define the starting LED position (LED #1) and ending LED position for each channel/controller.
- **Acceptance Criteria:**
  - The customer can specify the physical location of LED #1 (e.g., "left side of garage")
  - The customer can specify the total LED count for the channel
  - The system validates LED count against the connected controller
  - Configuration persists across app sessions

REQ-2.1.2: Multi-Channel Support

- **Priority:** P1 (High)
- **Description:** The system SHALL support multiple channels/controllers per property.
- **Acceptance Criteria:**
  - The customer can configure 1–N channels per property
  - Each channel maintains independent LED mapping
  - Channels can be linked for synchronized control
  - The system displays aggregate LED count across all channels

---

2.2 Architectural Segment Mapping

REQ-2.2.1: Segment Definition

- **Priority:** P0 (Critical)
- **Description:** The system SHALL allow the customer to define distinct architectural segments along their roofline with precise LED boundaries.
- **Acceptance Criteria:**

- The customer can create segments with start/end LED numbers
- Supported segment types:
  - `run` — horizontal or angled straight section
  - `corner` — direction change point (90° or other angles)
  - `peak` — triangular apex point
  - `column` — vertical section
  - `connector` — transition between sections
- Each segment stores:
  - Segment ID (unique identifier)
  - Segment name/label
  - Segment type
  - Start LED index
  - End LED index
  - LED count (derived)
  - Anchor points (see REQ-2.2.2)

REQ-2.2.2: Anchor Point Identification

- **Priority:** P0 (Critical)
- **Description:** The system SHALL identify and store anchor points within segments that represent key architectural features.
- **Acceptance Criteria:**
  - Anchor points include:
    - Corners (direction changes)
    - Peaks (highest points of gables)
    - Segment boundaries
    - User-defined accent points
  - Each anchor point stores:
    - LED index position
    - Anchor type (corner, peak, boundary, custom)
    - Optional label
  - The system auto-calculates default anchors based on segment type:
    - `peak` → center LED is anchor
    - `corner` → midpoint LED is anchor
    - `run` → start and end LEDs are anchors

REQ-2.2.3: Directional Flow Tracking

- **Priority:** P1 (High)
- **Description:** The system SHALL track the directional flow of LEDs to understand the physical path of the light run.
- **Acceptance Criteria:**
  - The system records direction indicators:
    - Left-to-right / right-to-left
    - Upward / downward
    - Toward street / away from street
  - Directional data is used for:
    - Chase animation direction
    - Gradient calculations

        - Symmetry analysis

REQ-2.2.4: Segment Visualization

- **Priority:** P1 (High)
- **Description:** The system SHALL provide a visual representation of segments overlaid on the customer's house photo.
- **Acceptance Criteria:**
    - Segments displayed as colored regions on the house image
    - Anchor points highlighted with distinct markers
    - LED numbers displayed at key positions
    - Interactive editing of segment boundaries
    - Real-time preview of segment changes

2.3 Individual LED Control

REQ-2.3.1: Per-LED Addressability

- **Priority:** P0 (Critical)
- **Description:** The system SHALL enable individual control of every LED in the installation.
- **Acceptance Criteria:**
    - Each LED independently controllable for:
        - Color (RGB/RGBW)
        - Brightness (0–255)
        - On/off state
    - Changes apply in real time to the physical device
    - Per-LED state persists in saved designs

REQ-2.3.2: Efficient LED Selection Interface

- **Priority:** P0 (Critical)
- **Description:** The system SHALL provide an intuitive, efficient interface for selecting and manipulating LEDs without requiring individual clicks.
- **Acceptance Criteria:**
    - **Range Selection** — tap-drag to select an LED range (e.g., LEDs 28–45)
    - **Segment Selection** — one-tap to select an entire segment
    - **Pattern Selection** — select every Nth LED (e.g., every 3rd LED)
    - **Anchor Selection** — one-tap to select all anchor points
    - **Inverse Selection** — select all non-selected LEDs
    - **Smart Selection** — select by segment type (all corners, all peaks)
    - Visual feedback highlights selected LEDs
    - Selection count is displayed in the UI

REQ-2.3.3: Batch LED Operations

- **Priority:** P0 (Critical)
- **Description:** The system SHALL allow batch operations on selected LEDs.
- **Acceptance Criteria:**
    - Apply color to all selected LEDs
    - Apply brightness to all selected LEDs
    - Apply effect to all selected LEDs
    - Copy/paste LED configurations
    - Clear selection

    - Undo/redo operations (minimum 10 levels)

REQ-2.3.4: LED Visualization Modes

- **Priority:** P1 (High)
- **Description:** The system SHALL provide multiple visualization modes for LED editing.
- **Acceptance Criteria:**
    - **Strip View** — linear representation of the LED strip with zoom/pan
    - **Roofline View** — LEDs mapped onto the house photo overlay
    - **Grid View** — 2D grid for segment-based editing
    - Synchronized selection across all views
    - Toggle between views without losing state

2.4 Roofline Intelligence Integration

REQ-2.4.1: Profile-Aware Recommendations

- **Priority:** P0 (Critical)
- **Description:** The system SHALL use roofline configuration data to generate personalized pattern recommendations.
- **Acceptance Criteria:**
    - Lumina AI accesses the roofline configuration when generating suggestions
    - Recommendations adapt to:
        - Number and position of peaks
        - Number and position of corners
        - Total LED count
        - Segment layout complexity
    - AI responses include roofline-specific instructions
    - Example: "For your double-peak roofline, I recommend..."

REQ-2.4.2: Accent Point Utilization

- **Priority:** P0 (Critical)
- **Description:** The system SHALL automatically utilize anchor/accent points when generating designs.
- **Acceptance Criteria:**
    - When the customer requests an accented design (e.g., "green with red accents"):
        - Primary color applied to run segments
        - Accent color applied to anchor points (peaks, corners)
    - Accent distribution is configurable:
        - Peaks only
        - Corners only
        - Peaks and corners
        - Custom anchor selection
    - Preview shows accent placement before applying

REQ-2.4.3: Architecture-Specific Pattern Library

- **Priority:** P1 (High)
- **Description:** The system SHALL categorize and filter patterns based on roofline architecture compatibility.
- **Acceptance Criteria:**
    - Patterns tagged with compatible architecture types:
        - Ranch (flat/minimal peaks)
        - Gabled (single peak)

        - Multi-gabled (multiple peaks)
        - Complex (mixed architecture)
    - Pattern library filters by the customer's architecture type
    - Incompatible patterns show a warning or adaptation suggestions

---

2.5 Spacing & Symmetry Logic

REQ-2.5.1: Downlighting Spacing Algorithm

- **Priority:** P0 (Critical)
- **Description:** The system SHALL calculate optimal LED spacing for downlighting patterns that maintains visual symmetry.
- **Acceptance Criteria:**
    - The customer specifies desired spacing (e.g., every 3rd LED, every 5th LED)
    - The algorithm calculates spacing per segment to ensure:
        - Equal visual spacing within each segment
        - Anchors (corners, peaks) always lit regardless of spacing
        - First and last LED of each segment always lit
    - Spacing adjusts automatically when segments have different lengths
    - Preview shows calculated spacing before applying

REQ-2.5.2: Symmetry Analysis

- **Priority:** P1 (High)
- **Description:** The system SHALL analyze and optimize designs for visual symmetry.
- **Acceptance Criteria:**
    - The system identifies a symmetry axis (typically center peak)
    - Symmetry modes:
        - **Mirror** — left side mirrors right side
        - **Radial** — patterns radiate from center
        - **None** — asymmetric design allowed
    - Symmetry warnings when a design is unbalanced
    - Auto-correct option to enforce symmetry

REQ-2.5.3: Brightness Gradient Patterns

- **Priority:** P1 (High)
- **Description:** The system SHALL support patterns with varied brightness levels (e.g., 1 bright, 3 dim).
- **Acceptance Criteria:**
    - The customer defines a brightness pattern sequence (e.g., `[255, 80, 80, 80]`)
    - Pattern repeats across a segment respecting spacing rules
    - Anchor points can override the brightness pattern
    - Preview shows brightness variation clearly

REQ-2.5.4: Segment-Aware Spacing

- **Priority:** P0 (Critical)
- **Description:** The system SHALL maintain consistent visual spacing across segments of varying lengths.
- **Acceptance Criteria:**
    - Given the customer wants "every 4th LED lit"
    - The system calculates per-segment:
        - Segment A (28 LEDs): LEDs 1, 8, 15, 22, 28 lit (adjusted for clean distribution)

        - Segment B (36 LEDs): LEDs 1, 10, 19, 28, 36 lit (adjusted for segment length)
    - Result: visually consistent spacing despite different segment lengths
    - Algorithm prioritizes:
        1. Anchor points always lit
        2. Segment boundaries always lit
        3. Even distribution between anchors

2.6 Design Persistence & Application

REQ-2.6.1: Design Save/Load

- **Priority:** P0 (Critical)
- **Description:** The system SHALL persist all design data including roofline-specific configurations.
- **Acceptance Criteria:**
    - A saved design includes:
        - Design name and metadata
        - Per-LED color/brightness states
        - Effect configurations
        - Roofline configuration reference
        - Timestamp and version
    - Designs load correctly on app restart
    - Designs sync across devices via the Lumina cloud

REQ-2.6.2: Design Application

- **Priority:** P0 (Critical)
- **Description:** The system SHALL apply designs to physical LED hardware with accurate reproduction.
- **Acceptance Criteria:**
    - Design converts to a valid controller JSON payload
    - Per-LED designs use the controller's individual LED control (`"i"` array)
    - Segment-based designs use standard segment control
    - Apply confirms success/failure to the customer
    - Rollback option if apply fails

REQ-2.6.3: Design Preview

- **Priority:** P1 (High)
- **Description:** The system SHALL provide an accurate preview before applying a design.
- **Acceptance Criteria:**
    - AR overlay shows the design on the house photo
    - Preview animates effects in real time
    - Preview matches physical result within acceptable tolerance
    - Preview is available without affecting the current device state

---

3. User Experience Requirements

3.1 Onboarding & Setup Flow

REQ-3.1.1: Guided Roofline Setup

- **Priority:** P0 (Critical)
- **Description:** The system SHALL provide a guided wizard for initial roofline configuration.

- **Acceptance Criteria:**
  - Step-by-step wizard with a progress indicator
  - Steps:
    1. Upload/capture house photo
    2. Trace the roofline path on the photo
    3. Mark segment boundaries
    4. Identify segment types
    5. Set anchor points
    6. Validate against the device's LED count
    7. Save the configuration
  - Skip option for advanced users
  - Tutorial videos/animations for each step

REQ-3.1.2: LED Identification Assistant

- **Priority:** P1 (High)
- **Description:** The system SHALL assist the customer in identifying LED positions on their physical installation.
- **Acceptance Criteria:**
  - "Find LED" mode that:
    - Lights up specific LED numbers on command
    - Runs a chase animation to show LED direction
    - Flashes segment boundaries
  - The customer can adjust LED mapping based on visual confirmation
  - Supports "mark as corner" while watching physical lights

3.2 Editing Experience

REQ-3.2.1: Touch-Optimized Interface

- **Priority:** P0 (Critical)
- **Description:** The system SHALL provide touch-optimized controls for mobile editing.
- **Acceptance Criteria:**
  - Minimum touch target size: 44×44 points
  - Pinch-to-zoom on the LED strip view
  - Two-finger pan for navigation
  - Long-press for context menus
  - Haptic feedback on selections

REQ-3.2.2: Undo/Redo System

- **Priority:** P1 (High)
- **Description:** The system SHALL maintain an undo/redo history for design edits.
- **Acceptance Criteria:**
  - Minimum 10 undo levels
  - Undo/redo buttons always visible
  - Keyboard shortcuts on tablet/desktop
  - History survives view changes within a session

REQ-3.2.3: Auto-Save

- **Priority:** P1 (High)
- **Description:** The system SHALL auto-save design progress to prevent data loss.
- **Acceptance Criteria:**
  - Auto-save every 30 seconds during editing

    - Auto-save when the app backgrounds
    - Draft recovery on app crash
    - Clear indication of save status

---

4. Integration Requirements

4.1 Lumina AI Integration

REQ-4.1.1: Natural Language Design Commands

- **Priority:** P0 (Critical)
- **Description:** The system SHALL accept natural language design commands that leverage roofline data.
- **Acceptance Criteria:**
    - Example commands understood:
        - "Create a Christmas pattern with green and red accents on the peaks"
        - "Make a downlighting pattern with every 4th light on"
        - "Apply Chiefs colors to my roofline with the red on corners"
    - The AI parses:
        - Primary/accent colors
        - Target segments/anchors
        - Spacing requirements
        - Effect preferences
    - Response includes a design preview

REQ-4.1.2: Contextual Suggestions

- **Priority:** P1 (High)
- **Description:** The system SHALL provide contextual design suggestions based on roofline architecture.
- **Acceptance Criteria:**
    - Suggestions adapt to:
        - The customer's specific roofline shape
        - Current season/holidays
        - Customer preferences from profile
        - Time of day
    - Suggestion cards show a roofline-aware preview
    - One-tap to apply a suggested design

4.2 Schedule Integration

REQ-4.2.1: Design-Based Scheduling

- **Priority:** P1 (High)
- **Description:** The system SHALL allow scheduling of custom designs.
- **Acceptance Criteria:**
    - Saved designs are available in the schedule action picker
    - The schedule can specify:
        - Design to apply
        - Brightness override
        - Duration
    - Scheduled designs apply correctly via controller timers

---

5. Technical Requirements

5.1 Controller Protocol Support

REQ-5.1.1: Individual LED Protocol

- **Priority:** P0 (Critical)
- **Description:** The system SHALL use the controller's individual LED control protocol for per-LED designs.
- **Acceptance Criteria:**
  - Uses the controller JSON API with `"i"` array format:

    ```
    {
      "seg": [{
        "i": [0, [255,0,0], 5, [0,255,0], 10, [0,0,255]]
      }]
    }
    ```

  - Supports both indexed and ranged formats
  - Handles RGBW devices correctly
  - Payload size optimization for large LED counts

REQ-5.1.2: Segment Protocol

- **Priority:** P0 (Critical)
- **Description:** The system SHALL use the controller's segment protocol for segment-based designs.
- **Acceptance Criteria:**
  - Creates multiple controller segments matching roofline segments
  - Segment boundaries align with architectural segments
  - Effects apply per-segment with correct parameters

5.2 Performance Requirements

REQ-5.2.1: Responsive Editing

- **Priority:** P1 (High)
- **Description:** The system SHALL maintain responsive performance during design editing.
- **Acceptance Criteria:**
  - LED selection responds within 100ms
  - Color changes preview within 200ms
  - Design save completes within 2 seconds
  - AR preview maintains 30fps minimum

REQ-5.2.2: Large Installation Support

- **Priority:** P1 (High)
- **Description:** The system SHALL support installations with up to 1000 LEDs.
- **Acceptance Criteria:**
  - UI remains responsive with 1000 LEDs
  - Payload generation handles 1000 LEDs
  - Preview renders 1000 LEDs smoothly

---

6. Data Model Requirements

6.1 Roofline Configuration Schema

```
RooflineConfiguration {
  String id;
  String userId;
  List<RooflineSegment> segments;
  int totalPixelCount;
  DateTime createdAt;
  DateTime updatedAt;
}

RooflineSegment {
  String id;
  String name;
  SegmentType type;  // run, corner, peak, column, connector
  int startLedIndex;
  int endLedIndex;
  int ledCount;
  List<AnchorPoint> anchors;
  SegmentDirection direction;
}

AnchorPoint {
  int ledIndex;
  AnchorType type;  // corner, peak, boundary, custom
  String? label;
}
```

6.2 Design Schema

```
CustomDesign {
  String id;
  String userId;
  String name;
  List<ChannelDesign> channels;
  String? rooflineConfigId;
  DesignType type;  // per-led, segment-based, pattern-template
  Map<String, dynamic> metadata;
  DateTime createdAt;
  DateTime updatedAt;
}

ChannelDesign {
  int channelId;
  List<LedColorGroup> colorGroups;
  int? effectId;
  int speed;
  int intensity;
  int brightness;
}
```

```
LedColorGroup {
  int startLed;
  int endLed;
  List<int> color;  // [R, G, B] or [R, G, B, W]
}
```

---

7. Success Metrics

Metric	Target	Measurement
Roofline setup completion rate	>80%	Customers completing the full setup wizard
Design creation time	<5 min	Average time to create first custom design
Design accuracy	>95%	Customer satisfaction with preview vs. physical result
AI command success rate	>90%	Natural language commands correctly interpreted
User retention	+20%	Increase in daily active users after feature launch

---

8. Dependencies

Dependency	Description	Status
Controller firmware	v0.14+ with individual LED support	Available
Lumina cloud	Cloud storage for configurations	Implemented
House photo upload	Customer photo or stock image	Implemented
Roofline editor	Interactive polyline drawing	Implemented
AR preview system	Animated overlay on house	Implemented

---

9. Out of Scope (v1.0)

The following features are explicitly out of scope for the initial release:

1. 3D roofline visualization
2. Automatic roofline detection from photo
3. Multi-property management
4. Design sharing marketplace
5. Third-party controller support (non-Nex-Gen)
6. Offline design editing

---

10. Revision History

Version	Date	Author	Changes

1.0	Jan 2026	Nex-Gen LED LLC Product Team	Initial requirements document
1.0	Apr 2026	Nex-Gen LED LLC Product Team	Brand and terminology refresh; content unchanged

---

Appendix A: Example Roofline Configuration

```
Home: 200 LED Installation
Channel 1: LEDs 1-200

Segments:
1. Garage Left Run (LEDs 1-28)
   - Type: run
   - Direction: left-to-right
   - Anchors: [1, 28]

2. Garage Corner Extension (LEDs 29-36)
   - Type: corner
   - Direction: toward-street
   - Anchors: [29, 33, 36]

3. Driveway Peak Left (LEDs 37-67)
   - Type: run
   - Direction: upward
   - Anchors: [37, 67]

4. Driveway Peak Center (LED 67)
   - Type: peak
   - Anchors: [67]

5. Driveway Peak Right (LEDs 68-97)
   - Type: run
   - Direction: downward
   - Anchors: [68, 97]

6. Front Door Approach (LEDs 98-138)
   - Type: run
   - Direction: toward-door
   - Anchors: [98, 138]

7. Front Door Section (LEDs 139-165)
   - Type: run
   - Direction: toward-street
   - Anchors: [139, 152, 165]

8. Front Facade Right (LEDs 166-200)
   - Type: run
   - Direction: left-to-right
   - Anchors: [166, 200]
```

```
Total Anchors: 16 key positions for accent lighting
```

Appendix B: Downlighting Spacing Algorithm Example

Input:

- Segment: 40 LEDs (index 98–138)
- Anchors at: 98, 118, 138
- Customer request: "Every 4th LED lit"

Algorithm:

1. Identify anchor zones: [98–118], [118–138]
2. Zone 1: 21 LEDs, need ~5 lit LEDs + anchors
3. Zone 2: 21 LEDs, need ~5 lit LEDs + anchors
4. Calculate: Zone 1 spacing = 21/5 ≈ 4.2 → LEDs 98, 102, 106, 110, 114, 118
5. Calculate: Zone 2 spacing = 21/5 ≈ 4.2 → LEDs 118, 122, 126, 130, 134, 138

Output: Lit LEDs: 98, 102, 106, 110, 114, 118, 122, 126, 130, 134, 138 Result: Visually even spacing with anchors always lit

---